

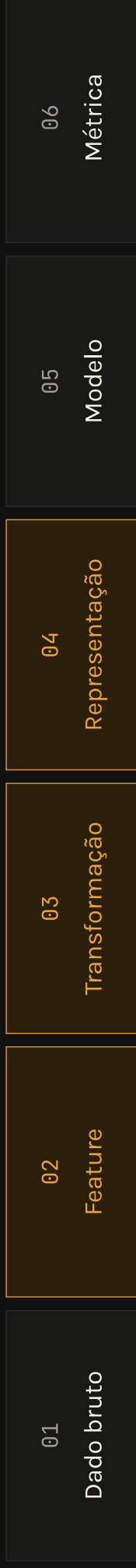
## 5.4

# Feature engineering: criação, codificação e escalonamento

Do absoluto básico — o que é uma feature — até target encoding, transformações de distribuição e transformer próprio no Pipeline.

# Machine learning não é só escolher o algoritmo

A representação dos dados influencia o resultado tanto quanto o modelo. Esta é a cadeia que a aula percorre:



Os quatro passos em destaque são os que você controla nesta aula. O modelo fica constante justamente para que a diferença na métrica venha da representação, e não do algoritmo.

## 01 OBJETIVOS

# No fim desta aula, você vai conseguir

01

Explicar o que é observação, variável, feature e target, e classificar o tipo de cada coluna.

03

Escolher entre Min-Max, Standard e Robust olhando a distribuição da coluna.

05

Identificar vazamento de dados em uma transformação e corrigir a ordem das etapas.

02

Criar uma feature a partir de uma hipótese escrita antes do código.

04

Codificar categóricas com one-hot, ordinal e target encoding, sabendo o custo de cada uma.

06

Medir o ganho de cada feature contra um baseline e descartar as que não pagam.

07

Montar um Pipeline com criação, imputação, codificação e escalonamento na ordem correta — e escrever um transformer próprio quando o sklearn não tem o que você precisa.

## O que ficou da aula passada

O **Pipeline** guarda a ordem das etapas e garante que o **fit** só veja o treino.

O **ColumnTransformer** dá caminhos separados para colunas numéricas e categóricas.

Imputação, codificação e escalonamento viraram um objeto só, serializável e reproduzível.

Hoje esse mesmo Pipeline ganha um primeiro passo: a criação das colunas que não existem no CSV.

```
pre = ColumnTransformer([\n    ("num", num_pipe, num_cols),\n    ("cat", cat_pipe, cat_cols),\n])\n\nmodel = Pipeline([\n    ("pre", pre),\n    ("clf", LogisticRegression(max_iter=1000))\n])
```

# As duas horas, em ordem

00-08

Objetivo, objetivos de aprendizagem e recapitulação da 5.3

08-20

Parte 1 — O que é uma feature: observação, variável, target, tipos

20-32

Parte 2 — Por que criar features, e quando a criação piora o modelo

32-38

Parte 3 — Feature engineering não é criar colunas aleatórias

38-55

Parte 4 — Escalonamento: Min-Max, Standard, Robust e a comparação

55-75

Parte 5 — Codificação: one-hot, ordinal, target encoding e vazamento

75-85

Avançado — binning, interações, distribuição e transformer próprio

85-95

Parte 6 — Titanic: experimento controlado, validação e Pipeline

95-100

Quiz — cinco perguntas

100-115

Exercício guiado — notebook completo em doze passos

115-120

Erros comuns, desafio de casa e documentação

# O que é uma feature?

Observação, variável, coluna, feature, target. Cinco palavras que costumam ser usadas como sinônimos e não são.

# Anatomia de uma tabela de dados

| nome  | idade | altura | peso | cidade   | sexo | comprou |
|-------|-------|--------|------|----------|------|---------|
| Ana   | 34    | 1,62   | 58   | Recife   | F    | 1       |
| Bruno | 51    | 1,78   | 92   | Curitiba | M    | 0       |
| Célia | 27    | 1,70   | 64   | Recife   | F    | 1       |

## OBSERVAÇÃO

Uma linha. Uma unidade do mundo real sobre a qual queremos prever algo: uma pessoa, um pedido, um passageiro. A tabela acima tem três.

## VARIÁVEL / COLUNA

Uma característica medida em todas as observações. "Variável" é o conceito; "coluna" é como ela aparece no arquivo. Sete aqui.

## VALOR

Uma célula: o cruzamento de uma observação com uma variável. 1,62 é o valor de altura para Ana.

# Feature e target: entrada e saída

## FEATURE • X

Uma variável usada como **entrada** do modelo. É o que está disponível no momento da previsão.

idade, altura, peso, cidade, sexo

Nenhuma coluna é feature por natureza. "Idade" é feature quando prevemos compra, e é target quando prevemos idade a partir de outros dados. O papel vem do problema, não do arquivo.

Uma pergunta resolve os casos difíceis: esse valor já existe no instante em que preciso prever? Se não existe, não pode ser feature.

## TARGET • Y

A variável que queremos **prever**. No treino ela é conhecida; na hora de usar o modelo, não.

comprou · Survived

```
Y = df["comprou"]           # target
X = df.drop(columns="comprou") # features

# feature engineering acontece
# sempre do lado do X
```

# Quais dessas colunas podem ser features?

| COLUNA       | SERVE COMO FEATURE?       | POR QUÊ                                                     |
|--------------|---------------------------|-------------------------------------------------------------|
| idade        | Sim, direto               | Numérica, comparável, disponível na previsão                |
| altura, peso | Sim — e melhor combinadas | Separadas funcionam; juntas viram IMC, que resume as duas   |
| cidade       | Sim, depois de codificar  | Catagórica sem ordem; cardinalidade pode ser alta           |
| sexo         | Sim, depois de codificar  | Binária: uma coluna 0/1 basta                               |
| nome         | Cru, não. Extraído, sim   | Quase único por pessoa; o título dentro dele é reutilizável |
| id_cliente   | Não                       | Identificador: o modelo decora em vez de generalizar        |

Três respostas possíveis para qualquer coluna: entra direto, entra transformada, ou não entra. A terceira também é uma decisão de feature engineering.

# O tipo da variável decide o tratamento

## NUMÉRICA CONTÍNUA

Qualquer valor num intervalo: altura, tarifa, idade.

Tratamento: imputação e escalonamento

## NUMÉRICA DISCRETA

Contagens inteiras: filhos a bordo, número de pedidos.

Tratamento: numérica, ou faixas

## BINÁRIA

Dois estados: sexo, assinante, tem cabine.

Tratamento: uma coluna 0/1

## CATEGÓRICA NOMINAL

Categorias sem ordem: cidade, porto de embarque, cor.

Tratamento: one-hot

## CATEGÓRICA ORDINAL

Categorias com ordem real: baixo/médio/alto, escolaridade.

Tratamento: ordinal, com ordem declarada

## TEXTO E DATA

Nome, endereço, data de nascimento.

Tratamento: extrair antes de usar

# Por que criar features?

O dataset original guarda a informação. Raramente a apresenta na forma que o modelo consegue usar.

# Por que o dado bruto costuma não bastar

## A INFORMAÇÃO ESTÁ IMPLÍCITA

O que importa não é peso nem altura isolados, é a razão entre eles. A razão não está em nenhuma coluna.

## O MODELO TEM LIMITES DE FORMA

Um modelo linear soma contribuições. Ele não multiplica, não divide e não inverte colunas por conta própria.

## DADOS SÃO FINITOS

Com milhões de linhas, um modelo flexível aprende a interação sozinho. Com 891 linhas, não. A feature entrega a interação de graça.

## O FORMATO NÃO É NUMÉRICO

"Recife" não é um número. Sem codificação, a coluna simplesmente não entra no modelo.

Feature engineering é reescrever a informação que já existe numa forma que o modelo escolhido consegue aproveitar.

# Cinco criações que aparecem em quase todo projeto

| DE                 | PARA         | O QUE A NOVA COLUNA ENTREGA                                                        |
|--------------------|--------------|------------------------------------------------------------------------------------|
| idade              | faixa etária | Permite efeito não linear: criança, adulto e idoso com comportamentos distintos    |
| data de nascimento | idade        | Uma data não é comparável entre gerações; a idade é                                |
| preço e quantidade | preço total  | O produto entre colunas, que um modelo linear não calcula sozinho                  |
| altura e peso      | IMC          | Uma razão com significado clínico, comparável entre pessoas de tamanhos diferentes |
| nome completo      | título       | Transforma texto quase único em categoria reutilizável                             |

Repare no padrão: toda linha combina, resume ou extrai. Nenhuma inventa informação que não estava no dado.

# As quatro features desta aula, e suas hipóteses

| FEATURE       | COMO É CRIADA     | HIPÓTESE                                                           | QUANDO PODE PIORAR                                                |
|---------------|-------------------|--------------------------------------------------------------------|-------------------------------------------------------------------|
| FamilySize    | SibSp + Parch + 1 | Família média embarca junto no bote; família muito grande se perde | Se o efeito for monotônico, a soma já estava em SibSp e Parch     |
| IsAlone       | FamilySize == 1   | Viajar sozinho muda a prioridade no resgate                        | Redundante com FamilySize: mesma informação, menos resolução      |
| FarePerPerson | Fare / FamilySize | A tarifa é do bilhete, não da pessoa; dividir revela a classe real | Divisão por zero e valores extremos se o denominador variar muito |
| Title         | extraído de Name  | O tratamento social carrega idade, sexo e status ao mesmo tempo    | Títulos raros com 2 ou 3 linhas viram regra decorada              |

A quarta coluna é a que quase ninguém escreve — e é a que separa criação de feature de chute com nome de coluna.

# Quando criar uma feature piora o modelo

## REDUNDÂNCIA

A nova coluna repete o que outra já dizia. Em modelo linear, os coeficientes ficam instáveis e a interpretação se perde.

## RUÍDO

A coluna não tem relação com o alvo, mas por acaso ajuda no treino. O modelo ajusta a esse acaso e perde no teste.

## DIMENSIONALIDADE

Muitas colunas para poucas linhas: cada observação fica isolada no espaço e o modelo decora em vez de generalizar.

## VAZAMENTO

A feature usa informação do alvo, do conjunto inteiro ou do futuro. A métrica sobe muito e a previsão real não funciona.

Por isso a validação vem junto. Criar é fácil; o trabalho é provar que valeu.

PARTE 3 DE 6 • MÉTODO

# Não é criar colunas aleatórias

Toda feature começa por uma hipótese e termina por uma decisão registrada.

# O ciclo de uma feature

01

## Hipótese

Uma frase em português

02

## Feature

Código dentro do Pipeline

03

## Treinamento

Mesmo modelo, mesma semente

04

## Validação

Cruzada, nunca no teste

05

## Métrica

Média e desvio entre dobras

06

## Decisão

Mantém ou descarta, por escrito

## 0 LIMIAR VEM ANTES DO NÚMERO

Defina o ganho mínimo aceitável antes de rodar. Sem isso, qualquer oscilação entre dobras passa a parecer descoberta.

## CINQUENTA COLUNAS SEM VALIDAÇÃO

Gera overfitting, treino mais lento, modelo difícil de explicar e um pipeline que ninguém consegue manter três meses depois.

PARTE 4 DE 6 • ESCALONAMENTO

# Colocar as colunas na mesma escala

Min-Max, Standard e Robust. Três fórmulas e uma decisão: o que fazer com o valor que está longe de todos os outros.

# Idade 20, salário 5000: por que isso é um problema

Duas pessoas, duas colunas. O KNN decide quem é parecido com quem calculando distância entre pontos.

```
A = (idade 25, salário 3000)
B = (idade 45, salário 3100)
```

```
dist = sqrt(20^2 + 100^2)
      = sqrt(400 + 10000)
      = 102.0
```

```
# a idade contribui com 400
# o salário contribui com 10000
```

## O QUE ACONTECEU

Vinte anos de diferença viraram 4% da distância. Cem reais viraram 96%. Não porque salário importa mais, mas porque reais são numericamente maiores que anos.

## ONDE ISSO DÓI

Em qualquer modelo que use distância (KNN, K-Means, SVM-RBF, PCA) ou que otimize por gradiente: a coluna de maior amplitude domina o passo de atualização e a convergência fica lenta e enviesada.

## A CORREÇÃO

Reescrever as duas colunas numa escala comum, para que a comparação seja entre variações relativas e não entre unidades de medida.

# Quais modelos exigem escala

## EXIGEM – DISTÂNCIA OU GRADIENTE

KNN e K-Means  
SVM com kernel RBF  
PCA  
Ridge, Lasso e ElasticNet  
Regressão logística com regularização  
Redes neurais

## INDIFERENTES – CORTAM POR LIMIAR

Árvore de decisão  
Random Forest  
Gradient Boosting  
XGBoost e LightGBM  
Naive Bayes

Escalonar não estraga um modelo de árvore, só gasta tempo. Não escalonar quebra os da coluna esquerda em silêncio: o modelo treina, a métrica sai, e a feature de maior amplitude domina a decisão.

## Min-Max: trazer tudo para [0, 1]

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

### CADA TERMO

$x$  é o valor original.  $x_{\min}$  e  $x_{\max}$  são o menor e o maior valor observados **no treino**. O denominador é a amplitude.

O mínimo vira 0, o máximo vira 1, e o resto se distribui proporcionalmente entre os dois.

### USE QUANDO

Existe limite natural conhecido (pixel, percentual, nota) e os extremos são reais, não erro de medição.

EXEMPLO: IDADES 20, 30, 50

$\min = 20$      $\max = 50$     amplitude = 30

$20 \rightarrow (20-20)/30 = 0.00$

$30 \rightarrow (30-20)/30 = 0.33$

$50 \rightarrow (50-20)/30 = 1.00$

### SENSIBILIDADE A OUTLIER

Um único valor extremo define o máximo e comprime todo o resto perto de zero. Valor novo acima do máximo do treino sai de [0, 1].

# Standard: média zero, desvio um

$$z = (x - \mu) / \sigma$$

## CADA TERMO

$\mu$  é a média: soma dos valores dividida pela quantidade.

Subtrair a média **centraliza** a coluna em zero.

$\sigma$  é o desvio padrão: o tamanho típico do afastamento em relação à média. Dividir por ele **padroniza a escala**.

O resultado se lê direto:  $z = 2$  significa dois desvios acima da média.

EXEMPLO: 10, 20, 30, 40, 50

|                                        |                  |
|----------------------------------------|------------------|
| $\mu = 30$                             | $\sigma = 14.14$ |
| $10 \rightarrow (10-30)/14.14 = -1.41$ |                  |
| $20 \rightarrow (20-30)/14.14 = -0.71$ |                  |
| $30 \rightarrow (30-30)/14.14 = 0.00$  |                  |
| $40 \rightarrow (40-30)/14.14 = +0.71$ |                  |
| $50 \rightarrow (50-30)/14.14 = +1.41$ |                  |

## USE QUANDO

A distribuição é aproximadamente simétrica, ou o modelo é linear regularizado, SVM ou rede neural. É o padrão de fato.

## DOIS ESCLARECIMENTOS

Não existe intervalo fixo: o resultado pode ser -4 ou +9. E padronizar não torna a distribuição normal — só recentra e redimensiona.

# Robust: mediana e intervalo interquartil

$$x' = (x - \text{mediana}) / (Q3 - Q1)$$

## MEDIANA

O valor do meio da lista ordenada. Trocar o maior valor por um mil vezes maior não a move — diferente da média.

## Q1, Q3 E IQR

Q1 corta os 25% menores, Q3 os 25% maiores. O IQR é a distância entre eles: a amplitude dos 50% centrais.

## RESISTÊNCIA

Nem mediana nem IQR dependem dos extremos. A escala passa a ser definida pelo miolo dos dados.

## USE QUANDO

Há cauda longa ou extremos legítimos: preço, renda, tempo de sessão, tarifa. No Titanic, é o caso de Fare.

## CUIDADO

Não limita amplitude: o outlier continua longe, e isso é proposital. Se o modelo precisa de entrada limitada, robust sozinho não resolve.

# Os três no mesmo dado, com um outlier

Coluna de seis valores: 10, 12, 13, 14, 15 e 500. Um erro de digitação — ou uma tarifa de primeira classe.

| ORIGINAL | MIN - MAX | STANDARD | ROBUST |
|----------|-----------|----------|--------|
| 10       | 0,000     | -0,46    | -1,40  |
| 12       | 0,004     | -0,45    | -0,60  |
| 13       | 0,006     | -0,45    | -0,20  |
| 14       | 0,008     | -0,44    | 0,20   |
| 15       | 0,010     | -0,44    | 0,60   |
| 500      | 1,000     | 2,24     | 194,60 |

min 10 • max 500

Cabe em [0, 1] e perde a diferença entre os cinco primeiros valores.

$\mu$  94 •  $\sigma$  181,6

A média foi puxada para 94, acima de cinco dos seis valores. Todos eles viram -0,45.

mediana 13,5 • IQR 2,5

Mantém a variação dos cinco legível e deixa o outlier visível como outlier.

# Escalonamento no Pipeline

```
from sklearn.preprocessing import (
    MinMaxScaler, StandardScaler, RobustScaler)

num_pipe = Pipeline([
    ("imp", SimpleImputer(strategy="median")),
    ("sc", RobustScaler()),
])

model = Pipeline([("pre", pre), ("clf", clf)])

model.fit(X_train, y_train) # aprende os parametros e
model.score(X_test, y_test) # apenas aplica aqui
```

## O QUE O FIT GUARDA

Min-Max guarda min e max. Standard guarda  $\mu$  e  $\sigma$ . Robust guarda mediana, Q1 e Q3. Poucos números por coluna, estimados só no treino e reaplicados no teste.

## COMO ESCOLHER SEM CHUTAR

Coloque o escalador no grid de busca — `param_grid = {"pre__num__sc": [StandardScaler(), RobustScaler()]}` — e deixe a validação cruzada decidir.

PARTE 5 DE 6 • CODIFICAÇÃO

# Variáveis categóricas

Do texto ao número sem inventar uma ordem que não existe — e sem deixar o alvo entrar pela porta de trás.

# O modelo não entende "male"

## 0 QUE ACONTECE SEM CODIFICAR

```
sex      country
male     Brazil
female   Brazil
female   USA

clf.fit(X, y)
# ValueError: could not convert
# string to float: 'male'
```

Todo modelo do sklearn faz aritmética: soma, multiplica, calcula distância. Texto não entra nessa conta.

Daí as três estratégias a seguir: one-hot, ordinal e target encoding. Cada uma resolve o erro de tipo com um custo diferente.

## A SOLUÇÃO INGÊNUA, E POR QUE FALHA

```
Brazil → 0
USA     → 1
Japan   → 2

# o modelo linear agora assume:
# Japan > USA > Brazil
# e (Brazil + Japan) / 2 == USA
```

Trocar texto por números inteiros resolve o erro e cria uma afirmação falsa: que existe ordem e distância entre países.

# Qual codificação usar, e quando

| SITUAÇÃO                                    | MÉTODO                              | CUSTO                         |
|---------------------------------------------|-------------------------------------|-------------------------------|
| Ordem real na variável                      | OrdinalEncoder                      | 1 coluna                      |
| Sem ordem, até ~15 categorias               | OneHotEncoder                       | n colunas                     |
| Muitas categorias, relação clara com o alvo | TargetEncoder                       | 1 coluna · risco de vazamento |
| Muitas categorias, frequência importa       | count / frequency encoding          | 1 coluna                      |
| Cardinalidade explosiva (CEP, ID, URL)      | FeatureHasher                       | k colunas fixas · colisões    |
| Árvore com muitas categorias                | ordinal ou suporte nativo do modelo | 1 coluna                      |

Cardinalidade é o número de categorias distintas. Sex tem 2, Embarked tem 3, Ticket tem mais de 600.

# One-hot encoding: uma coluna por categoria

## VANTAGENS

Não inventa ordem nem distância. Cada categoria ganha seu próprio coeficiente, o que torna o modelo linear fácil de interpretar.

## DESVANTAGENS

O número de colunas cresce com a cardinalidade. Com 600 categorias, viram 600 colunas quase todas zero — matriz esparsa, treino lento, overfitting fácil.

## TRÊS ARGUMENTOS QUE IMPORTAM

**handle\_unknown="ignore"**: sem ele, uma categoria inédita no teste levanta exceção em produção.

**drop="first"**: remove a coluna redundante que incomoda regressão linear sem regularização. Com Ridge, Lasso ou árvore, mantenha todas.

**min\_frequency**: categorias raras viram uma única coluna "infrequent".

| sex    |   | sex_female | sex_male |
|--------|---|------------|----------|
| male   | → | 0          | 1        |
| female | → | 1          | 0        |
| female | → | 1          | 0        |

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(
    handle_unknown="ignore",
    drop=None,
    min_frequency=10,
    sparse_output=False,
)
```

# Ordinal encoding: só quando a ordem existe

```
from sklearn.preprocessing import OrdinalEncoder

enc = OrdinalEncoder(
    categories=[["baixo", "medio", "alto"]],
    handle_unknown="use_encoded_value",
    unknown_value=-1,
)

# baixo → 0.0    medio → 1.0    alto → 2.0
# a distancia entre baixo e alto e 2:
# isso agora e uma afirmacao do modelo
```

## FAZ SENTIDO EM

Escolaridade, faixa de renda, nível de plano, grau de satisfação, Pclass. Em todos, trocar a ordem mudaria o significado.

## SEMPRE DECLARE A ORDEM

Sem `categories`, o `sklearn` ordena alfabeticamente: `alto=0`, `baixo=1`, `medio=2`. A ordem fica errada e nenhum erro é levantado.

## USOS ERRADOS

Cidade: `Recife=0`, `São Paulo=1`, `Curitiba=2` afirma que Curitiba é o dobro de São Paulo.

Cor, porto de embarque, sobrenome: mesma armadilha.

Exceção prática: em modelos de árvore, ordinal em nominal costuma ser inofensivo, porque a árvore corta por limiar e pode isolar cada valor.

# Target encoding: a média do alvo por categoria

| categoria | n   | media de y | codigo |
|-----------|-----|------------|--------|
| Miss      | 182 | 0.70       | 0.69   |
| Mrs       | 125 | 0.79       | 0.78   |
| Mr        | 517 | 0.16       | 0.16   |
| Master    | 40  | 0.57       | 0.56   |
| Rare      | 7   | 0.43       | 0.45   |

Cada categoria vira um número: a taxa de sobrevivência do grupo. Uma coluna só, qualquer cardinalidade.

$$(n^c \cdot \bar{y}^c + m \cdot \bar{y}) / (n^c + m)$$

A suavização puxa a média da categoria para a média global.  $n^c$  é o tamanho da categoria,  $\bar{y}^c$  a média dela,  $\bar{y}$  a média geral e  $m$  o peso da suavização.

Repare em Rare: 7 linhas, média 0,43, código 0,45 — puxado para o global. Já Mr, com 517 linhas, quase não se move.

Sem suavização, uma categoria com duas linhas receberia 0,0 ou 1,0 — e o modelo trataria isso como certeza.

## Como o target encoding vaza

### ERRADO

```
media = df.groupby("cidade")["y"].mean()
df["cidade_enc"] = df["cidade"].map(media)

X_tr, X_te, ... = train_test_split(df, ...)
```

Duas falhas ao mesmo tempo. A média foi calculada antes do split, então o alvo do teste entrou na feature do treino. E a média de cada categoria inclui a própria linha.

Caso extremo: uma categoria com uma linha só. A média dela é o y daquela linha. A feature passa a ser uma cópia do alvo.

Resultado: **acurácia altíssima na validação e queda no mundo real.**

### CORRETO

```
from sklearn.preprocessing import TargetEncoder

te = TargetEncoder(smooth="auto", cv=5)

model = Pipeline(["te", te), ("clf", clf)])
cross_val_score(model, X_train, y_train, cv=5)
```

Dentro do Pipeline, o encoder só vê o treino de cada dobra. E o `fit_transform` do TargetEncoder faz validação cruzada interna: cada linha recebe a média calculada nas **outras** dobras, não na dela.

Por isso `fit_transform` e `transform` devolvem valores diferentes aqui — e isso é correto, não bug.

# Quando há centenas de categorias

## FREQUENCY ENCODING

Cada categoria vira a sua contagem no treino. Uma coluna, não usa y, e captura popularidade — que muitas vezes é o que importa.

```
freq = X_train["c"].value_counts()  
X["c_freq"] = X["c"].map(freq)
```

## AGRUPAR AS RARAS

Tudo abaixo de um corte de frequência vira "Outros". Reduz colunas e evita que uma categoria com três linhas se torne uma regra.

```
OneHotEncoder(  
    min_frequency=10)
```

## HASHING

Número fixo de colunas, sem dicionário em memória. Aceita colisão entre categorias em troca de escala e processamento em streaming.

```
FeatureHasher(  
    n_features=16)
```

No Titanic isso aparece em duas colunas: Cabin, com centenas de valores quase únicos, e Ticket. Nenhuma entra crua — entra a primeira letra, ou a contagem.

# Faixas e termos de interação

## DISCRETIZAÇÃO · FAIXA ETÁRIA

```
KBinsDiscretizer(  
    n_bins=5,  
    encode="onehot-dense",  
    strategy="quantile",  
)
```

Transforma idade em cinco faixas. O modelo linear passa a poder dizer: criança sobe, adulto desce, idoso fica no meio — relação que uma única coluna numérica não expressa.

Custo: perde a diferença entre 31 e 39 anos se caírem na mesma faixa.

Árvores encontram cortes e interações sozinhas. Estes dois passos existem sobretudo para dar a modelos lineares o que eles não conseguem expressar.

## INTERAÇÃO · PRODUTO ENTRE COLUNAS

```
PolynomialFeatures(  
    degree=2,  
    interaction_only=True,  
    include_bias=False,  
)
```

Cria o produto de cada par de colunas. Permite que o efeito de uma feature dependa da outra — por exemplo, classe e sexo agindo em conjunto.

Custo: com 20 colunas, gera 190 produtos. A regularização passa a trabalhar contra você.

# Transformações de distribuição

Escalonar move o eixo. Estas quatro mudam a forma da distribuição.

| TRANSFORMAÇÃO                                          | EFEITO                                                                | RESTRIÇÃO                                               |
|--------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------|
| <code>np.log1p</code>                                  | Comprime cauda longa à direita; ganhos multiplicativos viram aditivos | Aceita zero, não aceita negativo                        |
| <code>PowerTransformer</code> <code>box-cox</code>     | Estima o expoente $\lambda$ que mais aproxima a coluna de uma normal  | Exige valores estritamente positivos                    |
| <code>PowerTransformer</code> <code>yeo-johnson</code> | Mesma ideia, funcionando com zero e negativos                         | $\lambda$ é parâmetro aprendido: estime só no treino    |
| <code>QuantileTransformer</code>                       | Força distribuição uniforme ou normal pelo ranking dos valores        | Destrói distâncias originais; precisa de amostra grande |

No Titanic, Fare é o candidato óbvio: assimetria forte, zero presente, máximo em 512. `log1p` resolve a maior parte com uma linha.

# Escrever seu próprio transformer

```
from sklearn.base import BaseEstimator, TransformerMixin

class FeaturesTitanic(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self # nada aprendido aqui

    def transform(self, X):
        X = X.copy()
        X["FamilySize"] = X["SibSp"] + X["Parch"] + 1
        X["IsAlone"] = (X["FamilySize"] == 1).astype(int)
        X["FarePerPerson"] = X["Fare"] / X["FamilySize"]
        X["Title"] = (X["Name"]
                      .str.extract(r",\s*([^\.\,]+)\.", expand=False)
                      .str.strip())
        return X

model = Pipeline([
    ("feat", FeaturesTitanic()),
    ("pre", pre),
    ("clf", LogisticRegression(max_iter=1000)),
```

Com a criação dentro do Pipeline, a validação cruzada avalia a feature junto com o modelo — e o mesmo objeto salvo em disco reproduz a feature em produção.

## O CONTRATO

`fit` aprende e devolve `self`. `transform` aplica e devolve os dados. Herdar de `BaseEstimator` dá `get_params` de graça, e é o que permite usar o `transformer` no `GridSearchCV`.

## ATALHO E REGRA

Sem estado, `FunctionTransformer(f)` basta. Se o `transform` precisa de algo estimado (mediana, mapa de frequência, categorias vistas), guarde no `fit` com `suffix_` e use no `transform`.

PARTE 6 DE 6 • APLICAÇÃO

# Titanic, do baseline ao experimento

Sete colunas de partida, quatro features novas, e um protocolo de comparação que não se engana.

# As sete colunas do baseline

| COLUNA   | TIPO                              | TRATAMENTO                             |
|----------|-----------------------------------|----------------------------------------|
| Pclass   | Catagórica ordinal (1, 2, 3)      | Ordinal, ou numérica direta            |
| Sex      | Binária                           | One-hot (ou 0/1)                       |
| Age      | Numérica contínua · 177 faltantes | Imputar pela mediana, depois escalonar |
| SibSp    | Numérica discreta                 | Direta                                 |
| Parch    | Numérica discreta                 | Direta                                 |
| Fare     | Numérica contínua · cauda longa   | Robust, ou log1p + Standard            |
| Embarked | Catagórica nominal · 2 faltantes  | Imputar pela moda, depois one-hot      |

Ficam de fora: PassengerId e Ticket (identificadores), Name e Cabin (entram só depois de extração).

# O experimento controlado

01

Modelo base  
7 colunas do CSV

02

Métrica  
CV 5 dobras, média  
e desvio

03

Uma feature nova  
Só uma por rodada

04

Treinar de novo  
Mesmo modelo,  
mesma semente

05

Métrica  
Mesmas dobras

06

Comparar  
Ganho maior que o  
limiar?

## UMA VARIÁVEL POR VEZ

Se você adiciona duas features e troca o escalador na mesma rodada, o ganho não tem dono. A comparação só é informativa quando muda uma coisa.

## OLHE O DESVIO, NÃO SÓ A MÉDIA

Ganho de 0,3 ponto com desvio de 2 pontos entre dobras não é ganho. É variação de amostra com nome de melhoria.

# Cada conjunto tem uma função

## TREINO

Aprende a transformação e o modelo. É aqui que se calculam mediana, média, desvio, categorias e médias por classe.

## VALIDAÇÃO

Escolhe: qual feature fica, qual escalador, qual hiperparâmetro. Pode ser olhada muitas vezes — é para isso que existe.

## TESTE

Avalia uma vez, no fim. Serve para estimar o desempenho em dados novos — e perde essa função no momento em que influencia uma decisão.

Se você testa dez conjuntos de features no teste e escolhe o melhor, o número obtido não é mais uma estimativa honesta: é o máximo de dez sorteios. Com dados novos, ele cai.

Na validação cruzada, a transformação é reajustada dentro de cada dobra. O Pipeline faz isso automaticamente; à mão, quase nunca sai certo.

```
# cada dobra: fit no treino da dobra,  
# transform na validacao da dobra  
cross_val_score(  
    model,          # Pipeline inteiro  
    X_train, y_train,  
    cv=StratifiedKFold(5, shuffle=True,  
                        random_state=42),  
    scoring="accuracy",  
)
```

# A ordem das etapas, e por quê

01 Dados

02 **Feature engineering**  
criar colunas novas primeiro

03 Imputação  
inclui os faltantes das novas

04 Encoding  
sem faltante para atrapalhar

05 Scaling  
só nas colunas numéricas

06 Modelo

```
num_cols = ["Age", "SibSp", "Parch", "Fare",  
            "FamilySize", "FarePerPerson"]  
cat_cols = ["Pclass", "Sex", "Embarked", "Title"]  
  
num_pipe = Pipeline([  
    ("imp", SimpleImputer(strategy="median")),  
    ("sc", RobustScaler()),  
])  
cat_pipe = Pipeline([  
    ("imp", SimpleImputer(strategy="most_frequent")),  
    ("ohe", OneHotEncoder(handle_unknown="ignore",  
                           min_frequency=10)),  
])  
  
pre = ColumnTransformer([  
    ("num", num_pipe, num_cols),  
    ("cat", cat_pipe, cat_cols),  
])  
  
model = Pipeline([
```

05 QUIZ • 5 MIN

## Cinco perguntas antes do notebook

- 01 Por que salário e idade podem exigir escalonamento antes de entrar no mesmo modelo?
- 02 Quando o one-hot encoding é a escolha apropriada — e quando ele deixa de ser?
- 03 Por que o ordinal encoding pode ser perigoso numa coluna como "cidade"?
- 04 Qual é o problema de calcular o target encoding usando o dataset inteiro?
- 05 Por que não devemos escolher features olhando repetidamente o conjunto de teste?

As respostas estão nas notas do apresentador deste slide.

06 · EXERCÍCIO GUIADO · 15 MIN

# Notebook completo, doze passos

Titanic, scikit-learn, executável no Google Colab. Do CSV à tabela de ganho por feature.

## 06 PASSOS 1 A 4 • IMPORTAR, SEPARAR, CRIAR, DIVIDIR

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, RobustScaler
from sklearn.linear_model import LogisticRegression
from sklearn.base import BaseEstimator, TransformerMixin

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# 1. importar

y = df["Survived"]
# 2. separar X e y
X = df.drop(columns=["Survived", "PassengerId", "Ticket"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
# 4. dividir antes de qualquer fit
```

O passo 3 — criar features — vira o transformer do próximo slide, e por isso entra no Pipeline em vez de mexer no DataFrame aqui.

## 06 PASSOS 5 A 8 • TRANSFORMAR, CODIFICAR, ESCALONAR, TREINAR

```
class Features(BaseEstimator, TransformerMixin):    # 5. transformacao
    def __init__(self, usar=()): self.usar = usar
    def fit(self, X, y=None): return self
    def transform(self, X):
        X = X.copy()
        X["FamilySize"] = X["SibSp"] + X["Parch"] + 1
        X["IsAlone"] = (X["FamilySize"] == 1).astype(int)
        X["FarePerPerson"] = X["Fare"] / X["FamilySize"]
        X["Title"] = (X["Name"].str.extract(r',\s*([^\.]+)\.', expand=False).str.strip()
                      .replace(["Mlle", "Ms", "Miss"], "Miss").replace("Mme", "Mrs"))
        X["Title"] = X["Title"].where(X["Title"].isin(["Mr", "Miss", "Mrs", "Master"]), "Rare")
        return X.drop(columns=["Name", "Cabin"])

def make_model(num_cols, cat_cols):
    num = Pipeline([("imp", SimpleImputer(strategy="median")),    # 7. scaling
                    ("sc", RobustScaler())])
    cat = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),    # 6. encoding
                    ("ohe", OneHotEncoder(handle_unknown="ignore", min_frequency=10))])
    pre = ColumnTransformer([("num", num, num_cols), ("cat", cat, cat_cols)])
    return Pipeline([("feat", Features()), ("pre", pre),
                    ("clf", LogisticRegression(max_iter=1000))])    # 8. treinar
```

## 06 PASSOS 9 A 12 · MEDIR, ADICIONAR, COMPARAR, INTERPRETAR

```
NUM = ["Age", "SibSp", "Parch", "Fare"]          # 9. baseline
CAT = ["Pclass", "Sex", "Embarked"]

s = cross_val_score(make_model(NUM, CAT), X_train, y_train, cv=cv)
ref = s.mean()
print(f"baseline {ref:.3f} +/- {s.std():.3f}")

LIMIAR = 0.002                                     # declarado ANTES de ver os numeros
candidatas = [("FamilySize", "num"), ("IsAlone", "num"),
              ("FarePerPerson", "num"), ("Title", "cat")]

num, cat = list(NUM), list(CAT)
for nome, tipo in candidatas:
    n2, c2 = (num + [nome], cat) if tipo == "num" else (num, cat + [nome])
    s = cross_val_score(make_model(n2, c2), X_train, y_train, cv=cv)
    ganho = s.mean() - ref
    print(f"{nome:>14} {s.mean():.3f} +/- {s.std():.3f} ganho {ganho:+.3f}")
    if ganho > LIMiar:
        num, cat, ref = n2, c2, s.mean()          # 12. interpretar e decidir
```

# Ganho medido por feature criada

Execução de referência: regressão logística, 5 dobras estratificadas, semente 42, acurácia média no treino. Limiar declarado: +0,002.

| CONJUNTO             | ACURÁCIA | GANHO  | DECISÃO               |
|----------------------|----------|--------|-----------------------|
| Baseline (7 colunas) | 0,792    | —      | referência            |
| + FamilySize         | 0,801    | +0,009 | mantém                |
| + IsAlone            | 0,800    | -0,001 | descarta · redundante |
| + FarePerPerson      | 0,805    | +0,004 | mantém                |
| + Title              | 0,819    | +0,014 | mantém                |

## LEITURA DO RESULTADO

Quatro features testadas, três aprovadas, 2,7 pontos de ganho acumulado. Title foi a mais forte — consistente com a hipótese de que o tratamento social resume idade, sexo e status.

## RESSALVA HONESTA

Com desvio entre dobras na casa de 0,02, um ganho de 0,004 é fraco. Reportar o desvio junto com a média é parte da resposta — não um detalhe.

## 07 ERROS COMUNS

# Dez armadilhas deste tema

### 01 • Escalar antes do split

Média e desvio do teste entram no treino e a métrica sobe sem motivo real.

### 03 • Target encoding sem dobra

Média calculada com a própria linha dentro, ou antes do split.

### 05 • Feature sem hipótese

Coluna criada por criar: aumenta a dimensionalidade e não explica nada.

### 07 • Transformação diferente no teste

Refazer o fit no teste, ou esquecer uma etapa. O Pipeline evita os dois.

### 09 • Ignorar outliers

Escalonar com Min-Max sem olhar a distribuição comprime todo o resto perto de zero.

### 02 • Usar o target para criar feature

Qualquer coluna derivada de y é o alvo entrando disfarçado de entrada.

### 04 • Ordinal em nominal

O modelo linear otimiza uma hierarquia criada por acidente alfabético.

### 06 • Escolher feature pelo teste

O teste deixa de ser estimativa honesta e vira o melhor de N tentativas.

### 08 • Informação do futuro

Coluna que só existe depois do evento previsto. Funciona no notebook, falha em produção.

### 10 • Ganho pequeno como prova

Melhoria de 0,2 ponto com desvio de 2 pontos entre dobras não decide nada.

# Melhore o modelo do Titanic com feature engineering

Crie pelo menos três features novas, cada uma precedida de uma hipótese escrita em uma frase. Meça o ganho de cada uma contra o baseline, com a mesma semente e as mesmas dobras, e registre as métricas.

Ao final, diga qual feature teve o maior impacto e por quê — ligando o resultado à hipótese que a motivou.

Nenhuma feature específica é obrigatória, e nenhuma é a resposta certa. O que está sendo avaliado é a formulação da hipótese e a honestidade da medição.

Toda criação dentro do Pipeline. O teste é aberto uma única vez, na última célula.

## CRITÉRIO DE ACEITE

- O Pipeline executa de cima a baixo sem erro
- Não existe vazamento evidente
- As três features estão justificadas por hipótese
- O baseline está registrado
- A comparação está documentada em tabela
- A métrica é adequada ao problema
- O código está organizado e legível

# Referência de tudo o que foi usado

## StandardScaler

[scikit-learn.org](#) · [preprocessing.StandardScaler](#)

## RobustScaler

[scikit-learn.org](#) · [preprocessing.RobustScaler](#)

## OrdinalEncoder

[scikit-learn.org](#) · [preprocessing.OrdinalEncoder](#)

## ColumnTransformer

[scikit-learn.org](#) · [compose.ColumnTransformer](#)

## Preprocessing data · guia 6.3

[scikit-learn.org](#) · [modules/preprocessing](#)

## MinMaxScaler

[scikit-learn.org](#) · [preprocessing.MinMaxScaler](#)

## OneHotEncoder

[scikit-learn.org](#) · [preprocessing.OneHotEncoder](#)

## TargetEncoder

[scikit-learn.org](#) · [preprocessing.TargetEncoder](#)

## Pipeline

[scikit-learn.org](#) · [pipeline.Pipeline](#)

## Compare the effect of different scalers

[scikit-learn.org](#) · exemplo com cauda longa

## 5.5

# Seleção de features: filtros, wrappers e embedded

ANOVA e informação mútua, RFE, e a seleção orgânica do Lasso e da Random Forest – com o custo computacional de cada abordagem na mesa.